REAL-TIME EMBEDDED SYSTEM
EXPERIMENT NO. 1

REAL-TIME SCORE BOARD FOR BASKETBALL GAME

Name: Lelis, Kathryna T.

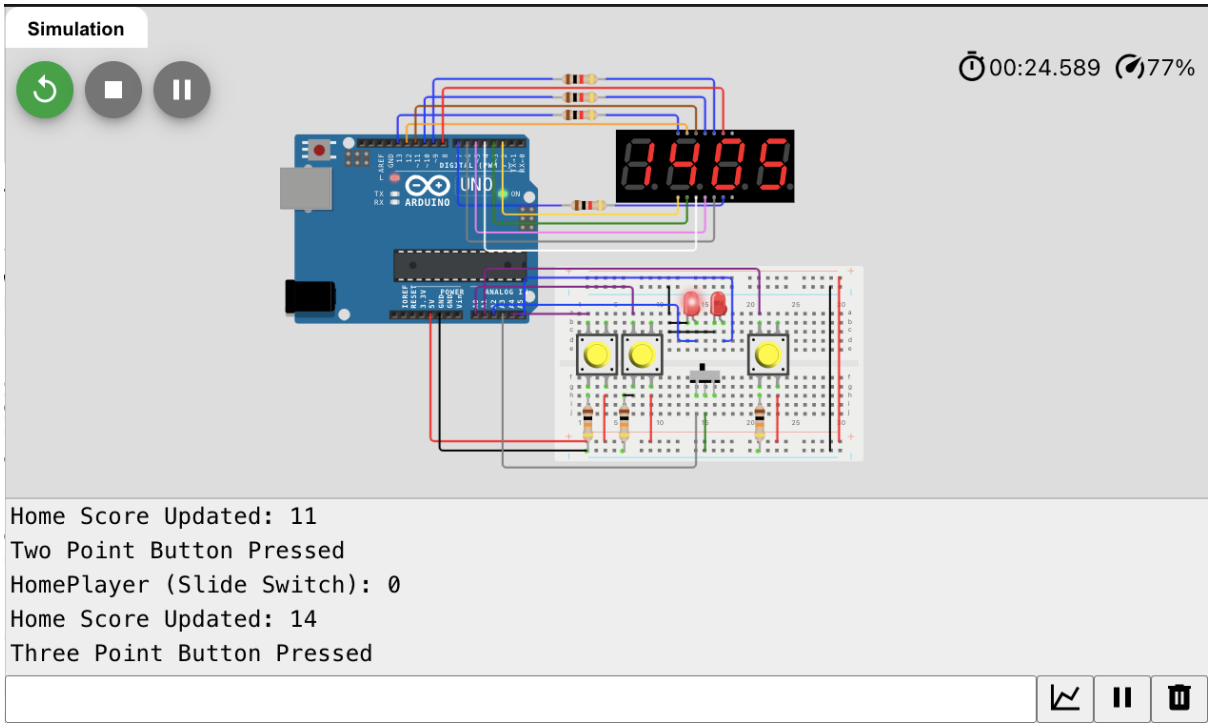
Course/Section: CPE161P-4/C1

Group No.: INDIVIDUAL

Date of Performance: September 10, 2024
Date of Submission: September 14, 2024

Cyrel O. Manlises, Ph.D.
Instructor

DISCUSSION



Experiment 1 (Real-Time Scoreboard For basketball game) as same as any other experiment begun in creating simulation for easy debugging and checking of potential errors to avoid breaking the physical components. The whole circuit diagram is shown above. I've decided to use 3 buttons for the points, 2 led for team

identifier, slide switch to change state between teams, and 4 digit seven segment display to display real time scoring.This components except the slide switch are not new to my knowledge therefore it s not that challenging to work with. Although im new with the slide switch, it wasnt that hard to integrate yet I misunderstand its pinout at first therefore causing errors on physical implementation. Which I later realize the mistake when debugging.

```
void setup() {
  // Initialize segment pins
  for (int i = 0; i < 7; i++) {
    pinMode(SegmentPins[i], OUTPUT);
  }

  // Initialize digit pins
  for (int i = 0; i < 4; i++) {
    pinMode(digitPins[i], OUTPUT);
    digitalWrite(digitPins[i], HIGH); // Turn off digits initially
  }

  // Initialize button pins
  pinMode(OnePointButton, INPUT);
  pinMode(TwoPointButton, INPUT);
  pinMode(ThreePointButton, INPUT);

  // Player input pin for HomePlayer
  pinMode(HomePlayer, INPUT_PULLUP);   // HomePlayer pin should be pulled up as there no external resistor
  pinMode(GuestPlayer, OUTPUT);        // LedPIN as Output
  pinMode(HomeLed, OUTPUT);            // LedPIN as Output

  // Start Serial Monitor for debugging
  Serial.begin(9600);
}
```

After creating the diagram, coding the logic and flow of the system is integrated. Initializing all components and variables to be used in the whole system. In the second image we can see that I also initialize serial monitor, this is used for debugging and checking button and variable states.

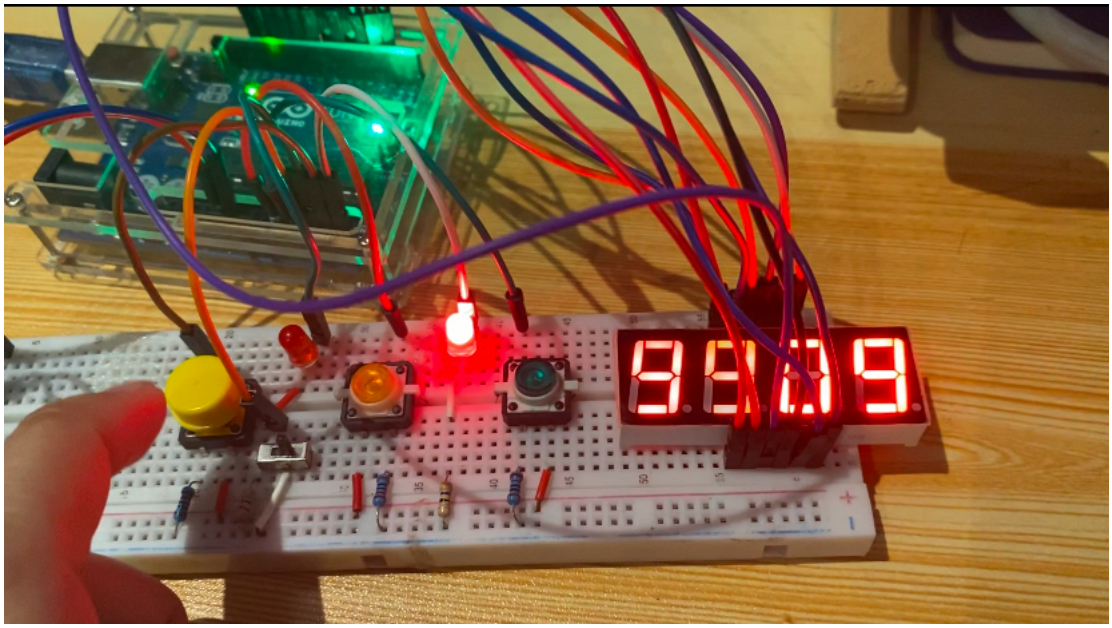
```
void checkButton() {
  // Button Debounce
  if (millis() - lastButtonCheck > debounceDelay) {
    if (digitalRead(OnePointButton) == HIGH) {
      updateScore(1); //1 point
      lastButtonCheck = millis();
      Serial.println("One Point Button Pressed");
    }
    if (digitalRead(TwoPointButton) == HIGH) {
      updateScore(2); //2 Point
      lastButtonCheck = millis();
      Serial.println("Two Point Button Pressed");
    }
    if (digitalRead(ThreePointButton) == HIGH) {
      updateScore(3); //3 Point
      lastButtonCheck = millis();
      Serial.println("Three Point Button Pressed");
    }
  }
}
```

```
void updateScore(int points) {
  // Debugging: Print button states
  int homePlayerState = digitalRead(HomePlayer);  // Get the state of the slide switch
  Serial.print("HomePlayer (Slide Switch): ");
  Serial.println(homePlayerState);

  // If the HomePlayer slide switch is LOW, score is updated for HomePlayer
  if (homePlayerState == LOW) {  // HomePlayer pressed (active)
    HomeScore = min(HomeScore + points, 99); // Update Home score
    Serial.print("Home Score: ");
    Serial.println(HomeScore);
  }
  // If the HomePlayer slide switch is HIGH, score is updated for GuestPlayer
  else {
    GuestScore = min(GuestScore + points, 99); // Update Guest Score
    Serial.print("Guest Score: ");
    Serial.println(GuestScore);
  }
}
```

I've also separated the scoring and updateScore display function, this is to ensure that the updating of score is updated immediately without flickering the seven-segment display. millis() function is also used in debounce

for the buttons, to create more seamless input of button within the system.



After all the code and system is working fine on the simulation, integrating it physical is the final step. The image above shows fully working system having 99 as its maximum score for both teams. Multiple testing and verifying of fully working function of the system is then done. With all objects checked, the real-time score board for the basketball game is created.

CONCLUSION

The successful completion of the Real-Time Basketball Scoreboard project demonstrates an in-depth understanding of how to use Arduino and basic electronics to create a functional system. This project points out the significance of debugging and testing ideas step by step to ensure that everything works as intended, first simulating the concept and then physically implementing it.

The main objectives, such as creating a smooth scoring system that refreshes instantly, were met. The system performed better and avoided troubles like display flickering by breaking tasks into methods like updateScore and using

capabilities like the millis() function for smooth button inputs.

Another benefit of this project was learning how circuits and programming interact. Solving issues, like the slide switch connecting mistake, was a phase in the method that tested the effectiveness of troubleshooting.

In summary, this project of Real-Time Basketball Scoreboard is a great example on how simple technology such as Arduino can be used for creating something useful and yet learning new programming and electronics skills.